

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе № 8
по дисциплине «Вычислительная математика»
Тема: Решение СЛАУ методом Якоби

Студентка гр. 4341

Преподаватель

Кочененко А.А.

Пуеров Г.Ю.

Санкт-Петербург

2026

Задание

В работе студенты должны найти решение системы линейных уравнений с n неизвестными, заданной матрицей коэффициентов и вектором свободных членов b , методом простых итераций. Выполнение работы включает следующие этапы:

1) с помощью преподавателя определить систему уравнений, которую нужно решить. Привести исходную систему к виду (2.3), пригодному для использования метода простых итераций;

2) задать необходимую точность получения результата (количество знаков мантиссы числа);

3) разработать программу решения задачи на языке C с использованием подпрограммы-функции MITER из файла MITER.CPP. Функция MITER имеет следующие параметры:

- a - матрица коэффициентов преобразованной к виду (2.3) системы уравнений размера $n \times n$, тип float;
- b - вектор свободных членов преобразованной системы размера n , тип float;
- x - полученный в результате проведения итераций вектор решения размера n , тип float;
- n - размер системы уравнений, тип int;
- $ntch$ - количество знаков после запятой в мантиссе результата, остающихся после округления, тип int, $0 \leq ntch \leq 6$;
- it - выходной параметр, равный количеству произведенных итераций, тип int.

В качестве значения функции типа возвращается одно из следующих значений:

- а) 0 - все нормально, получено решение ;
- б) 1 - не выполняются условия сходимости итерационного процесса;
- в) 2 - размер n ;
- г) 3 - значение $ntch$.

Константа должна быть задана при разработке головной программы аналогично тому, как это делается при выполнении лабораторной работы № 11

Теоретические положения

При решении системы линейных уравнений методом Якоби сначала проверяется размер подаваемой на вход матрицы, определить является ли она квадратной. После проверяется диагональное преобладание матрицы (одно из условий сходимости метода). Если диагональное преобладание отсутствует, можно попытаться переставить строки для его достижения, или может быть расхождение.

Система преобразуется к виду удобному для итераций диагональный элемент = оставшимся элементам с противоположным знаком и свободному члену

Система линейных уравнений приводится к форме, удобной для итерационного метода Якоби, где каждое уравнение выражается в виде:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} (b_i - \sum_{j \neq i} a_{ij} x_j^k), \text{ где } k - \text{номер итерации}$$

Расшифровка формулы:

1. Все элементы строки, кроме диагонального, делятся на диагональный элемент с противоположным знаком.
2. Свободный член также делится на диагональный элемент.
3. Диагональные элементы матрицы замещаются нулями, так как они больше не участвуют в вычислениях напрямую.

Начальное приближение – вектор правой части. На каждой итерации вычисляем новые значения приближения на основе предыдущих. Итерации продолжаются до тех пор, пока норма разности между текущим и предыдущим приближениями не станет меньше заданной точности *epsilon*.

$$\|x_{k+1} - x_k\| < \epsilon$$

Если процесс сходится, то мы нашли приближения с заданной точностью, если после заданного максимального числа итераций, то сходимость метода не достигнута.

Выполнение работы

1. `def parse_args()` – функция создаёт и обрабатывает аргументы командной строки, `cli`
2. `def validate_input(args)` – функция проверяет корректность входных данных, полученных из аргументов командной строки.
3. `def generate_diagonal_dominant (n, interval)` - функция генерирует расширенную матрицу диагонально-доминантную $A|b$. n – размер матрицы, `interval` – максимальное число интервала для случайной генерации.
4. `def prepare_jacobi_matrix(n, matrix, b, epsilon)` – функция преобразует матрицу для метода Якоби(см. теоретические положения).
5. `def jacobi_solve(n, epsilon, matrix, b)` – основная функция реализует метод Якоби
6. `compute_solution_diff_norm(my_solution, scipy_solution)` – функция вычисляет L2-норму (евклидову норму) разности между решением, полученным методом Якоби, и эталонным решением из библиотеки SciPy.
7. `def convert_epsilon(digit_epsilon)` – функция конвертирует представление числовое в $10^{-(digit_epsilon)}$.
8. `def exploration(n, interval, digit_epsilon)` – функция вычисляет на разных точностях кол-во итераций и строит зависимость количество итераций от точности *epsilon*.

Таблица 1 – тестирования метода Якоби на разных размерах матриц

Размер матрицы (n)	Норма разности (L2-норма)
10	2.10×10^{-7}
100	3.92×10^{-7}
500	2.26×10^{-8}
1000	8.79×10^{-8}

5000	2.13×10^{-7}
10000	2.14×10^{-8}

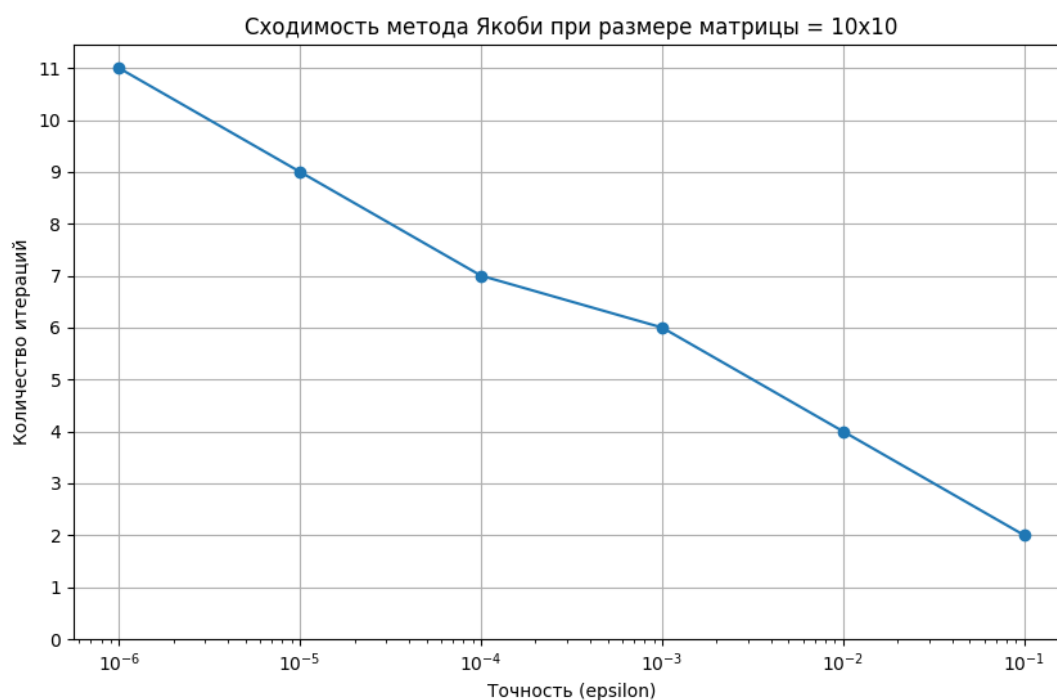


Рисунок № 1 – График зависимости количества итераций от точности

Выводы

В результате выполнения данной лабораторной работы был изучен алгоритм решения СЛАУ методом Якоби. Была исследована зависимость количества итераций от точности, построен соответствующий график. В консоль выводится сравнение полученного решения с эталонным решением библиотеки SciPy, а также норма разности между ними.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb8.py

```
import random
import scipy
import argparse
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator

def parse_args():
    parser = argparse.ArgumentParser(
        description="Linear system solver and iterations analysis using
method Jacobi",
        formatter_class=argparse.RawTextHelpFormatter,
        epilog="Example usage \n"
            "    Research mode:\n"
            "        python lb8.py research --n 10 --epsilon 6\n"
            "    Solution mode:\n"
            "        python lb8.py solve --n 10 --epsilon 6\n"
    )
    subparsers = parser.add_subparsers(dest='command', required=True)

    research_parser = subparsers.add_parser(
        'research',
        help="Compare matrix types and their conditioning"
    )

    research_parser.add_argument(
        "--n",
        type=int,
        required=True,
        help="Matrix dimension positive digit"
    )

    research_parser.add_argument(
        "--interval",
        type=int,
        required=False,
        default=100,
        help="Max random digit (default 100)"
    )

    research_parser.add_argument(
        "--epsilon",
        type=int,
        required=False,
        default=6,
        help="Last epsilon digit above 0 (default 6)"
    )

    jacobi_parser = subparsers.add_parser(
        'solve',
        help="Solve specific linear system"
    )

    jacobi_parser.add_argument(
        "--n",
        type=int,
        required=True,
        help="Matrix dimension positive digit"
```

```

    )

    jacobi_parser.add_argument(
        "--interval",
        type=int,
        required=False,
        default=10,
        help="Max random digit (default 10)"
    )

    jacobi_parser.add_argument(
        "--epsilon",
        type=int,
        required=True,
        help="Epsilon digit above 0"
    )
    return parser.parse_args()

def validate_input(args):
    if args.n < 3: raise ValueError("Matrix size must be above 3")
    if args.epsilon < 0: raise ValueError("Epsilon value must be above 0")
    if (args.command != "solve") and (args.command != "research"): raise
ValueError("Command not found try python lb8.py --help")
    if args.interval < 0 or args.interval > 1000000: raise
ValueError("Interval must be above 0 or below 1000000")

def generate_diagonal_dominant(n, interval):
    matrix = [[random.uniform(-interval, interval + 1) for _ in range(n)]
for _ in range(n)]
    for i in range(n):
        row_sum = sum(abs(matrix[i][j]) for j in range(n) if i != j)
        matrix[i][i] = row_sum + random.uniform(1, interval)
    b = [random.uniform(-interval, interval) for _ in range(n)]
    return matrix, b

def prepare_jacobi_matrix(n, matrix, b, epsilon):
    jacobi_matrix = []
    for i in range(n):
        if abs(matrix[i][i]) < epsilon:
            raise ValueError(f"Матрица вырождена, нулевой элемент в строке
{i}")
        row = []
        for j in range(n):
            if i != j: row.append(-matrix[i][j] / matrix[i][i])
            else: row.append(0.0)
        jacobi_matrix.append(row)
    initial_approach = [b[i] / matrix[i][i] for i in range(n)]
    return jacobi_matrix, initial_approach

def jacobi_solve(n, epsilon, matrix, b):
    jacobi_matrix, initial_approach = prepare_jacobi_matrix(n, matrix, b,
epsilon)
    prev_approach = initial_approach
    iteration = 1
    max_iter = 1000
    while iteration < max_iter:
        current_approach = []
        for i in range(n):
            sigma = 0.0
            for j in range(n):
                sigma += jacobi_matrix[i][j] * prev_approach[j]
            current_approach.append(initial_approach[i] + sigma)
        iteration += 1
        if max(abs(current - previous) for current, previous in
zip(current_approach, prev_approach)) < epsilon:
            return current_approach, iteration

```

```

        prev_approach = current_approach
        print(f"Достигнут лимит итераций {max_iter} точность не достигнута")
        return prev_approach, iteration

def compute_solution_diff_norm(my_solution, scipy_solution):
    """Вычисляет L2-норму разности между двумя решениями"""
    n = len(my_solution)
    diff = [my_solution[i] - scipy_solution[i] for i in range(n)]
    norm = sum(d ** 2 for d in diff) ** 0.5
    return norm

def convert_epsilon(digit_epsilon):
    return float(10 ** (-digit_epsilon))

def exploration(n, interval, digit_epsilon):
    iterations = []
    epsilons = []
    matrix, b = generate_diagonal_dominant(n, interval)
    for i in range(1, digit_epsilon + 1):
        epsilon = convert_epsilon(i)
        approach, iteration = jacobi_solve(n, epsilon, matrix, b)
        epsilons.append(epsilon)
        iterations.append(iteration)
        print(f"epsilon = {epsilon:.0e}, итераций = {iteration}")

    plt.figure(figsize=(10, 6))
    plt.plot(epsilons, iterations, marker='o')
    plt.xscale("log")
    plt.xlabel("Точность (epsilon)")
    plt.ylabel("Количество итераций")
    plt.title(f"Сходимость метода Якоби при размере матрицы = {n}x{n}")
    plt.grid(True)
    plt.gca().yaxis.set_major_locator(MultipleLocator(1))
    plt.ylim(bottom=0)
    plt.show()

def main():
    args = parse_args()
    validate_input(args)

    if args.command == "research":
        exploration(args.n, args.interval, args.epsilon)

    elif args.command == "solve":
        print("=" * 70)
        print(f"МЕТОД ЯКОБИ - решение СЛАУ размером {args.n}x{args.n}")
        print("=" * 70)

        matrix, b = generate_diagonal_dominant(args.n, args.interval)
        epsilon = convert_epsilon(args.epsilon)

        approach, iteration = jacobi_solve(args.n, epsilon, matrix, b)
        scipy_solution = scipy.linalg.solve(matrix, b)
        diff_norm = compute_solution_diff_norm(approach, scipy_solution)

        print(f"\n{'='*70}")
        print(f"СРАВНЕНИЕ РЕШЕНИЙ (первые {min(args.n, 20)} из {args.n})")
        print(f"{'='*70}")
        print(f"{'Index':<8}{'Approach (Jacobi)':<25}{'SciPy (LAPACK)':<25}{'|Diff|':<12}")
        print(f"{'-'*70}")

        max_display = min(args.n, 20)
        format_epsilon = f".{args.epsilon + 2}f"

        for i in range(max_display):

```

```

        diff = abs(approach[i] - scipy_solution[i])
        print(f"x{i+1:<4}  {approach[i]:{format_epsilon}}
{scipy_solution[i]:{format_epsilon}}      {diff:.2e}")

    if args.n > 20:
        print(f"... и еще {args.n - 20} переменных (не показаны)")

    print(f"{'='*70}")
    print(f"  Норма разности с SciPy (L2-норма): {diff_norm:.5e}")
    print(f"{'='*70}")

if __name__ == "__main__":
    main()

```